SimGate: An AI Agent That Runs Stress Testing of a Driving Policy

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—**

About 150 words, written last, with the final numbers. Four sentences: (1) the problem: stress testing an end-to-end driving policy in closed loop takes thousands of runs; by hand it is slow, and left unattended it fails silently; (2) what SimGate is: an AI agent that runs the whole campaign, choosing only from fixed menus that code checks and scripts carry out; (3) the evidence: the scale of the unattended campaign, the share of failed launches healed with no person, and that no planted invalid run was kept; (4) the main finding: where failures are rare, letting the agent choose the next tests confirms the hardest cases sooner than rules, random search or Optuna.

***Index Terms*—**

Four to six index terms covering autonomous vehicle testing, stress testing, LLM agents and closed-loop simulation.

I. Introduction

Paragraph 1: background and gap. Motivate accelerated stress testing of end-to-end driving policies [1], [2], now possible in closed loop on real recorded scenes with neural-reconstruction simulators [3]–[5]. State the gap: a campaign is thousands of runs that someone must choose, relaunch when they crash, and verify; left unattended it fails silently (use concrete early examples: a misconfigured model, a delay that never applied, missing results), and simulation results can themselves be flaky or nondeterministic [6]–[8]. Then the second half of the gap: handing the campaign to an AI agent that acts in a fully closed loop does not fix this by itself. An evaluation of an AI-scientist system found many experiments failing and hallucinated numbers in its papers [9]; such systems hide pitfalls that only full trace logs reveal [10]; coding agents make checks pass by deleting failing tests [11]; data-analysis agents finish without error yet answer the wrong question [12]. Checking has not kept pace with what agents produce [13].

Identify applicable funding agency here. If none, delete this.

Paragraph 2: approach and core contributions. Open with the approach: hand the whole campaign to an AI agent but never let it decide what counts: it only proposes from fixed menus, code checks every proposal and every run, and every failure is kept with its evidence. Then list the contributions, each naming where it is described and where it is validated. (1) An agent that runs stress testing of an end-to-end driving policy end to end and unattended, in a neural-reconstruction simulator (Sec. III; Sec. IV-C). (2) An action design that makes it trustworthy: every job is a contract with a fixed menu, checked by code and carried out by scripts; the recovery menu is checked exhaustively, and every kept run is verified against its request (Sec. III-A, III-C, III-D; Sec. IV-B). (3) Evidence that letting the agent choose the next tests confirms the hardest cases sooner where failures are rare, at an equal budget against rules, random search and Optuna (Sec. III-B; Sec. IV-D). (4) Results a person can check: every failure the agent handled with its evidence, a case card for every confirmed hardest case, and the agent's failure explanations checked against rules and people (Sec. IV-C, IV-E, IV-F). Do not claim to be the "first".

Paragraph 3: organization of the paper. One sentence per section. Sec. II reviews accelerated stress testing, LLMs in scenario generation and testing, and agents that act safely. Sec. III presents the framework around Fig. 1: its overview, the outer loop that plans a study and chooses the next runs, the inner loop that checks each run and heals failed ones, and how a study closes with triage and a report. Sec. IV describes the simulation setup and validates the framework: the validity of kept runs, self-healing in unattended operation, corner-case discovery across search methods, the reliability of failure triage, and the failures found in the policy under test. Sec. V concludes with limitations and future work.

II. Related Work

Three short paragraphs. (1) Accelerated and adaptive stress testing (NADE [1], D2RL [2], adaptive stress testing [14], [15], Bayesian optimization [16], [17]): they supply search algorithms; this paper studies who chooses the next test and how a campaign stays valid. (2) LLMs in scenario generation and testing (LLM-attacker [18], Teach-to-Crash [19], EvoDrive [20], PlannerForge [21], LEADE [22], and the IV precedents Chat2Scenario [23], EC-Detector [24] and ARISE [25]): one line each on what they do and how SimGate differs. (3) Agents that act safely (Simplex [26]–[28], shielding [29], runtime rules on agent actions [30], [31], self-healing pipelines [32]), and the evidence that agents running experiments or analyses on their own [33], [34] produce unreliable results [9], [10], with the auditable runtimes proposed against it [12]. Close with one sentence on what sets this paper apart.

III. Framework Architecture and Design

Opening paragraph. Three or four sentences: SimGate runs a campaign as two nested loops (Fig. 1); the agent proposes, code checks and acts. Then what each subsection covers: overview (III-A), outer loop (III-B), inner loop (III-C, III-D), triage and report (III-E).

A. Framework Overview

Present Fig. 1 as a whole: a brief comes in, a checked plan sets up a study, the outer loop picks each round of runs, the inner loop runs, checks and heals every run, kept outcomes go back to the goal check, and once the goal is met each failure is explained and a report answers the brief. Explain the colors: orange boxes are the agent, which only proposes; blue boxes are plain code, which checks every proposal and acts on it; grey is a person (who writes the brief, approves the plan, enacts a new rule and takes over a halted run). Define a campaign and a study (scenes, knobs, goal, budget). Then the design rule shared by every orange box: a contract with five parts (when code calls it, what it sees, what it may answer from a fixed menu or schema, who checks the answer, and who acts: a script, never the agent). Leave the concrete simulator and scenarios to Sec. IV-A.

B. Outer Loop: Planning and Choosing the Next Runs

The top band of Fig. 1, left to right. The planner turns the brief into scenes, knobs, rounds and a goal, and code checks the plan before a person approves it. The proposer (rules, a baseline, or the agent) picks each round; every proposed run is checked before it is queued (a candidate scene, legal knob values) and anything else is dropped. Kept outcomes return to the goal check: define a confirmed hardest case precisely (a setting whose failure rate is surely above one half, by a confidence interval over its repeats, at most one per scene), since Sec. IV counts them. List the search methods that can act as the proposer (rules, random search with repeats, Optuna [35] with repeats, and the agent alone or combined with either), all under the same gate and budget.

C. Inner Loop: Running and Checking Each Run

The upper row of the bottom band of Fig. 1. Each run passes checks before launch, runs in closed loop, and passes checks after it, in a fixed order; it is kept only if the requested scenario provably happened (read back from its logs) and its motion is physically plausible. Stress that code, not the model, decides that a run failed and names the reason. Define the outcome a kept run carries: a failure counts only if the policy is at fault after it takes over. Then the batch audit and how new checks are learned: the auditor flags runs that are wrong in the same way and proposes rules; code admits a rule only if it catches the flagged runs and fires on no clean run; a person enacts it; from then on it is checked on every run *(Fig. 1 does not show the auditor yet; add it when the figure is redrawn).*

D. Inner Loop: Healing Failed Runs

The lower row of the bottom band of Fig. 1, zoomed in twice. Fig. 2 zooms into that row: the five steps around the investigator, of which only the third is the agent. Fig. 3 zooms further into steps 2 and 3: what the agent is given and what it may return. Walk through them: the diagnose contract is called only on a failed run and sees the failure status, the config, the attempt count, error lines and a machine snapshot, with read-only tools. It answers one of five skills; for each, say what the recovery script does with it and when the validator accepts it. RE-RUN queues the same run again. CONFIGURE changes how the run executes (context length, scene file, where the traffic model runs), never what it measures. RESTART_CLEANUP takes down this run's containers, then reruns it. CLEANUP_ENV clears leftovers from the whole machine; it is the only fix allowed when the machine was not ready, once per run. HALT stops the run for a person; it is always accepted, and after a machine failure it stops the whole batch. Every skill but HALT needs an attempt left. Code restricts the choice before launch and through the attempt budget, while after launch every skill is allowed and the choice is the agent's (for example, halting every physically wrong run). Note that the tools are read-only by permission, not by instruction. End with the exhaustive check of every answer the agent could give. *Redraw Fig. 2 in the paper's style: title it by its role, the inner loop's investigator, and show only the five steps; campaign counts belong in Sec. IV-C, not in this figure.*

FIGURE TO ADD (Fig. 3, a further zoom into steps 2 and 3 of Fig. 2; it does not exist yet): a block diagram of the diagnose agent's input and output. Left, the input that code packs as JSON: the failure status, the config, the attempt count and its limit, up to 15 error lines, and the machine snapshot. Middle, the agent: one fresh call whose system prompt is the contract, with read-only tools (the run's log and folder, the container list, GPU memory, disk). Right, its output: one skill and its params, from a fixed schema with no free text; then the validator (accept or reject) and what the recovery script does for each skill. Along the bottom, one real failure traced through it, e.g. a GPU out-of-memory crash where the agent checked GPU memory and containers, answered RE-RUN, was accepted, and the run was kept on its second attempt. Agent orange, code blue, as in Fig. 1.

E. Closing a Study: Triage and Report

The right end of the top band of Fig. 1. Triage: for each failed run the agent sees frames before the failure and the metrics, and answers a cause from a fixed list and whether the policy was at fault; counts over these answers are computed by code. Report: the agent answers the brief by citing setting ids; code writes every number, and a report with a hand-written count is refused.

IV. Simulation Results

Opening paragraph. Three or four sentences: the setup (IV-A), then one claim per subsection: valid runs (IV-B), unattended self-healing (IV-C), faster corner-case discovery (IV-D), trustworthy triage (IV-E), findings on the policy (IV-F). No numbers here. *Add a small table mapping each subsection to the part of Fig. 1 it tests: IV-B, the checks before and after each run, plus the batch audit; IV-C, the investigator, validator and recovery; IV-D, the proposer and the goal check (only the proposer changes between methods); IV-E, triage; IV-F, the report, i.e. what the whole pipeline produced.*

A. Simulation Setup

Two parts. **Simulation:** the simulator and policy (AlpaSim [5], VaVAM [36], linear MPC tracking, replayed traffic; planning at 2 Hz and 10 Hz), the knobs, the test scenarios (studies mapped to Euro NCAP tests [37], [38]), repeats and run budgets per method, the metrics (confirmed hardest cases, runs to reach the goal, model time) and the statistical tests. **Computing environment:** one workstation (GPU model and memory, CPU, RAM, disk, operating system) running two simulations at once on its single GPU; the simulator in containers, with the added retiming and seeding hooks off by default; the agent's model and how it is called (one fresh headless call per decision, read-only tools, time per call); run time (simulated seconds and wall-clock per run at each planning rate, total GPU hours). *Confirm the exact hardware and software versions for the whole campaign; a short table is a common way to report them.*

B. Validity of Kept Runs

Test whether every kept run is real data, in both directions: bad runs must be caught and good runs kept. (1) Planted faults: in three campaigns, runs were broken on purpose in ways that still end normally (deleted or corrupted results, a requested setting silently dropped, the recorded human driving the whole run, no controller in the loop). Count the invalid runs kept with no gate, with the per-run gate, and with the batch audit added, and give an upper bound on the share that could still slip through. Say which faults only the batch audit catches and why (they look the same in every run of a batch). (2) False alarms: the physics check on clean runs (none should be thrown away) and on runs with silent motion faults (all should be caught). (3) Close by noting that every later result uses kept runs only. *Add a small table, generated from the data: invalid runs kept per planted-fault campaign with no gate, the per-run gate, and gate + audit. No figure is needed.*

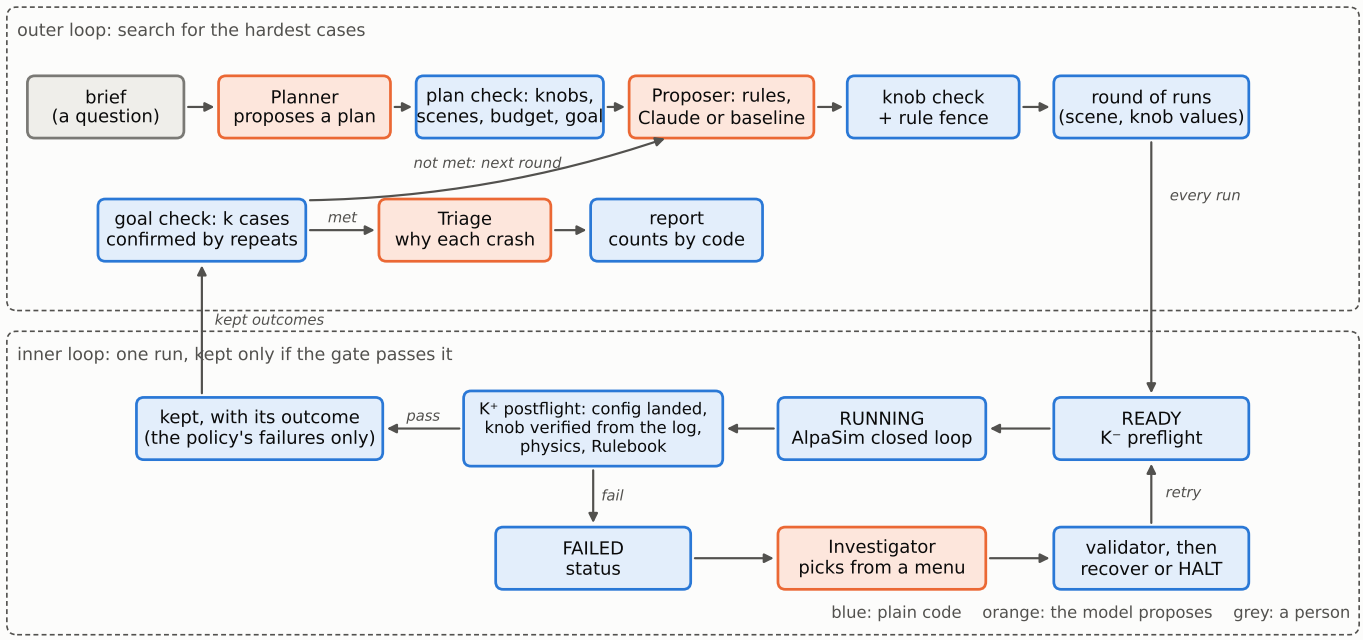


Fig. 1. The framework. Top band, the outer loop: a brief becomes a checked plan, a proposer picks each round of runs, and the goal check decides when to stop; each failure is then triaged and a report answers the brief. Bottom band, the inner loop: each run is checked before and after it runs and kept with its outcome or, if it fails, healed. Orange: the agent proposes; blue: plain code checks and acts; grey: a person.

C. Self-Healing in Unattended Operation

Test whether the agent keeps a long campaign running without a person, using every failed launch of the whole campaign. (1) Scale and outcome: runs asked for, runs kept on the first try, launches that failed and went to the agent, and how many of those were healed or stopped for a person (Fig. 4). (2) What the agent chose: for each kind of failure, the skill it picked and how the run ended; every answer passed the validator, and every retry ran exactly the settings asked for. (3) Honest accounting: most heals are out-of-memory crashes answered by a plain rerun, which a retry script would also do; the agent's value is in the rest, such as halting runs that failed the physics check instead of rerunning them and clearing a machine that was not ready. (4) Against a retry script: on the planted-fault campaigns, the same faults handled by a fixed script, by the agent without tools, and by the agent with read-only tools; compare valid runs kept, launches used and runs stopped, and note where the agent without tools fails (results missing despite a normal exit). Report the agent's time per decision. *Regenerate Fig. 4 from the final data before submission. Give (4) as a few numbers in the text, or a small table if space allows.*

D. Corner-Case Discovery Across Search Methods

Test whether letting the agent choose the next tests finds and confirms the hardest cases sooner, at an equal budget under the same gate. (1) The comparison: each method replaces only the proposer; studies, budgets, rounds and goal stay the same; each method runs several repeats. Report confirmed hardest cases at the budget and runs to reach the goal (Fig. 5, Table I). (2) Main result first: the three-knob study, where failures are rare, with five repeats and comparisons planned before the extra repeats were run; then lane keeping, a study the second AI version was not built on; then the car scenes and the pedestrian study, where failures are common. (3) The pattern: the agent's lead grows as failures get rarer, and where failures are common it has no edge (say so); Optuna alone does no better than random search with repeats; the agent restricted to the rules' candidates does well only where those candidates are good. Explain it by where each method looked: the rules start in the middle of each knob's range and step locally, random search and Optuna spread runs thin, the agent uses the physics stated in the brief and repeats what fails. (4) Counting decides the ranking: under a naive collision count (contacts begun before the hand-off, rear-ends by replayed traffic) the ranking changes; show it briefly. *Fig. 5 and Table I are placeholders with the three 10 Hz studies only; add the three-knob study and lane keeping, ordered from rare to common failures. Compute every test statistic with code.*

E. Reliability of Failure Triage

Check how far the agent's explanation of each failure can be trusted, against code and against people. (1) Against a rule in code that decides who closed the gap at the contact: on the collision failures of the car studies, how often the rule and triage agree on whether the ego or the other car was to blame, and where they disagree, including nearly identical contacts that triage labelled differently. (2) Against people: a person labels who caused each of 60 randomly drawn crashes without seeing the scorer or the triage; report agreement and Cohen's kappa with the scorer and with the triage. (3) A person marks each case card as making sense or not, including the hardest cases where triage and the scorer disagree. Close with what the failure types in Sec. IV-F can and cannot be trusted for. *Part (1) is done; (2) and (3) wait for the human labels and verdicts. Report (1)–(3) in one small table (check, cases, agreement, kappa); no figure is needed.*

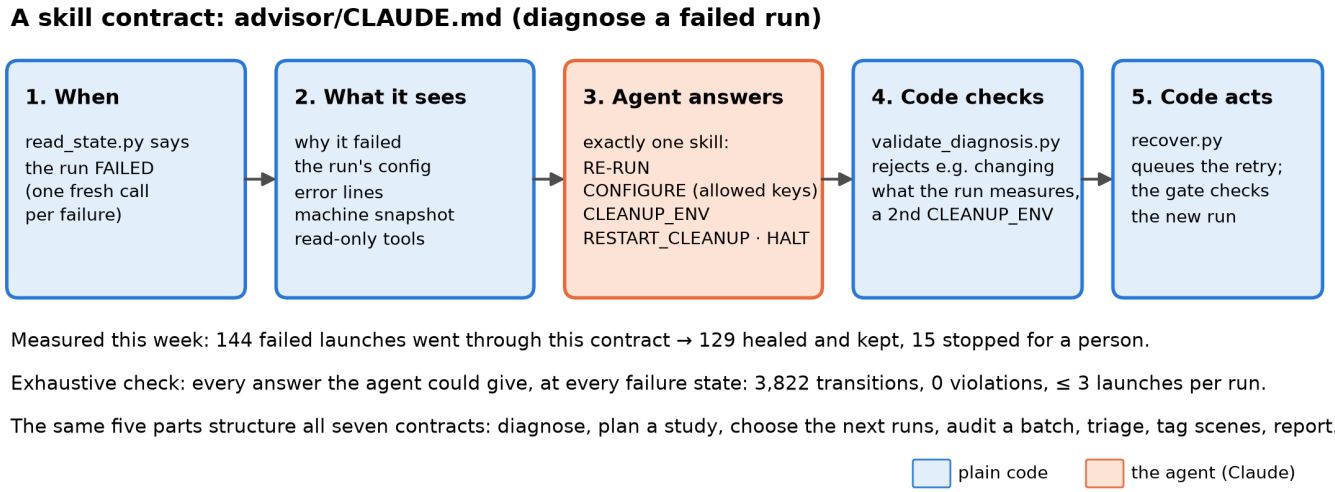


Fig. 2. A zoom into the lower row of Fig. 1: the five steps around the inner loop's investigator, which chooses how to recover a failed run. Only step 3 is the agent; code decides when it is called and what it sees, checks its answer, and acts on it. The planner, the proposer and triage follow the same five steps with their own inputs and menus.

Placeholder: the investigator's input and output (to be drawn).

Fig. 3. The investigator's input and output (to be drawn): what code packs for it, the agent's one call with read-only tools, and its answer from a fixed schema, followed by the validator and the recovery script; one real failure traced along the bottom.

TABLE I
SEARCH METHODS AT AN EQUAL BUDGET, THREE REPEATS EACH (10 HZ).

	Pedestrian	Car cutting in	Car ahead	Model
Method	runs to goal	confirmed	confirmed	s/arm
rules	25 (25, 25, 25)	2, 2, 2	0, 0, 0	–
hybrid (rules + AI)	24 (24, 28, 24)	3, 3, 3	2, 3, 3	87
AI picks	37 (28, 40, 37)	3, 2, 1	3, 1, 3	101
random + repeat	32 (31, 32, 32)	2, 0, 1	4, 1, 2	–
Optuna + repeat	31 (25, 31, 33)	4, 4, 0	1, 3, 1	–
Optuna + AI	21 (21, 20, 21)	2, 3, 1	1, 2, 4	97

Runs to goal: kept runs until 5 hardest settings were confirmed (median, then each repeat). Confirmed: of 5, per repeat. Contacts with a car the retiming held still are not counted. Model: median seconds of model calls per arm over the three studies.

F. Failures Found in the VaVAM Policy

A case study of what the campaign revealed about the policy under test, in four parts. (1) Hardest cases: for each case in Fig. 6, the setting that triggered it, what the ego did from the hand-off to the failure, and why it is the policy's fault. (2) Failure types: most failures are steering into another road user or leaving the road, few are failures to brake, although the studies were built around braking; the types come from the triage checked in Sec. IV-E. (3) The left drift: after the hand-off the ego moves steadily left of the human's path in nearly every run, never right; with the camera image and the plan mirrored it drifts right, so the bias is the model's, not the simulator's. Most hardest cases are this drift meeting traffic or the road edge. (4) The pedestrian stressor: retiming the pedestrian alone rarely causes failures; a faster ego at the hand-off does. Note that some pedestrian failures happen far from the recorded path, where rendering is less reliable, and that these are flagged. *Fig. 6 covers only part (1). Add a column-width figure for (3): the ego's lateral offset from the human's path against time since the hand-off, over all kept runs, with the mirrored runs drifting the other way. Give (2) as shares in the text; if space allows, a small table of failure types by study instead of a chart. No other table is needed here. Move the note printed inside Fig. 6 into its caption, and prefer cases that show the drift.*

V. CONCLUSION

Paragraph 1: summary. Restate the two main claims in a few sentences: an agent can run stress testing of a driving policy end to end, unattended, when each action is a checked choice from a fixed menu; and letting it choose the next tests pays off most where failures are rare. Say where the code and data are released.

Paragraph 2: limitations and future work. Pair each limitation with the direction that addresses it: one policy under test (test more end-to-end policies); replayed traffic does not react (use reactive traffic); few scenes for some maneuvers (add more scenario types); few repeats per method (run more repeats, and try other models as the agent). Also state the limitations with no direction yet: rendering is less certain far from the recording, and the human check of the scoring is still pending. End with the confirmed hardest cases reused as regression tests.

AI USE DISCLOSURE

Required by IEEE: state that the agent under study is Claude, and disclose any AI assistance in the code and the writing.

Cut the references to about 15. Keep the closest work (EvoDrive, PlannerForge, LLM-attacker, EC-Detector, ARISE), the testing roots (NADE, D2RL, AST), the tools used (AlpaSim, VaVAM, Optuna), and one or two each on unreliable agent results and safe agents.

REFERENCES

[1] S. Feng, X. Yan, H. Sun, Y. Feng, and H. X. Liu, "Intelligent driving intelligence test for autonomous vehicles with naturalistic and adversarial environment," *Nature Communications*, vol. 12, no. 1, p. 748, 2021.

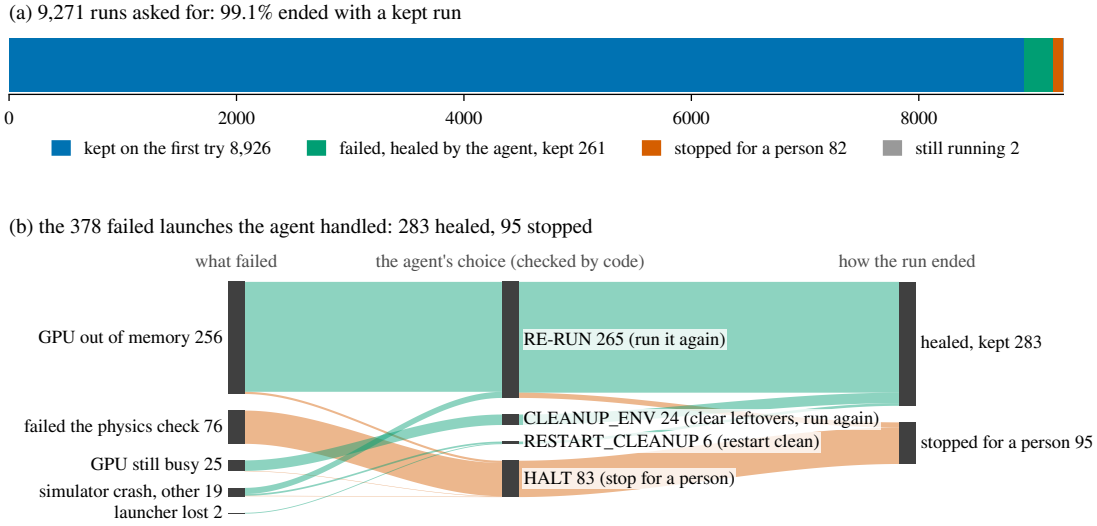


Fig. 4. Every run asked for, and every failed launch the agent handled: what failed, the skill it chose, how the run ended. Each failure is listed with its raw evidence in the released failure ledger.

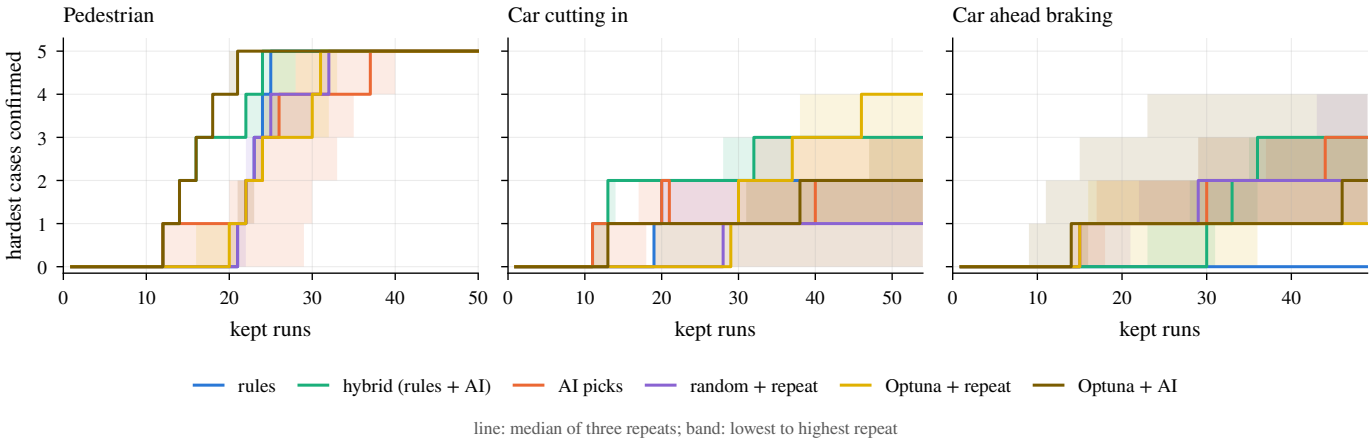


Fig. 5. Confirmed hardest cases against kept runs, per method: median and range over repeats.

[2] S. Feng, H. Sun, X. Yan, H. Zhu, Z. Zou, S. Shen, and H. X. Liu, "Dense reinforcement learning for safety validation of autonomous vehicles," *Nature*, vol. 615, no. 7953, pp. 620–627, 2023.

[3] W. Ljungbergh, A. Tonderski, J. Johnander, H. Caesar, K. Åström, M. Felsberg, and C. Petersson, "NeuroNCAP: Photorealistic closed-loop safety testing for autonomous driving," in *Proc. European Conference on Computer Vision (ECCV)*, 2024.

[4] H. Zhou, L. Lin, J. Wang, Y. Lu, D. Bai, B. Liu, Y. Wang, A. Geiger, and Y. Liao, "HUGSIM: A real-time, photo-realistic and closed-loop simulator for autonomous driving," 2024, arXiv:2412.01718.

[5] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. Garcia Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. Martinez Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, "AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving," https://github.com/NVlabs/alpasim, 2025.

[6] O. Osikowicz, P. McMinn, and D. Shin, "Empirically evaluating flaky tests for autonomous driving systems in simulated environments," in *Proc. 2nd International Flaky Tests Workshop (FTW), co-located with ICSE*, 2025.

[7] M. H. Amini, S. Naseri, and S. Nejati, "Evaluating the impact of flaky simulators on testing autonomous driving systems," *Empirical Software Engineering*, vol. 29, no. 2, p. 47, 2024.

[8] G. Chance, A. Ghobrial, K. McAreavey, S. Lemaignan, T. Pipe, and K. Eder, "On determinism of game engines used for simulation-based autonomous vehicle verification," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 11, pp. 20 538–20 552, 2022.

[9] J. Beel, M.-Y. Kan, and M. Baumgart, "Evaluating Sakana's AI scientist: Bold claims, mixed results, and a promising future?" 2025, arXiv:2502.14297.

[10] Z. Luo, A. Kasirzadeh, and N. B. Shah, "The more you automate, the less you see: Hidden pitfalls of AI scientist systems," 2025, arXiv:2509.08713.

[11] Z. Zhong, A. Raghunathan, and N. Carlini, "ImpossibleBench: Measuring LLMs' propensity of exploiting test cases," 2025, arXiv:2510.20270.

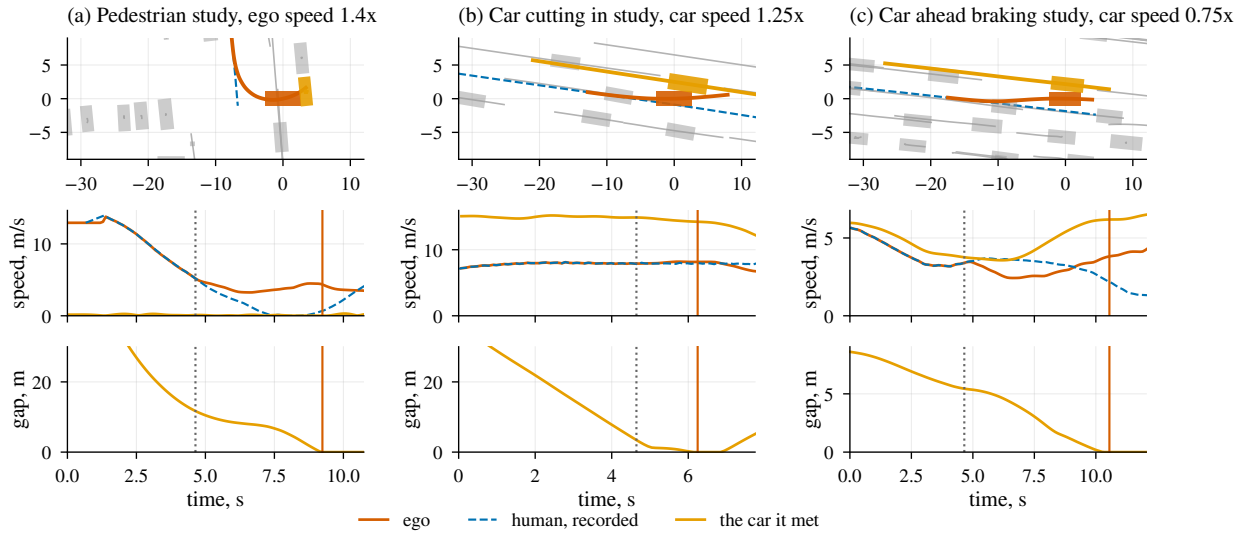
[12] H. Yan, L. Deng, Z. Wang, Y. Wang, and C. Liu, "Fail loudly: An auditable runtime for agentic data analysis," 2026, arXiv:2609.32528.

[13] B. Mo, "The age of AI agents demands a new scientific paradigm to sustain trustworthy science," 2026, arXiv:2607.26064.

[14] R. Lee, O. J. Mengshoel, A. Saksena, R. W. Gardner, D. Genin, J. Silbermann, M. Owen, and M. J. Kochenderfer, "Adaptive stress testing: Finding likely failure events with reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 69, pp. 1165–1201, 2020.

[15] M. Koren, S. Alsaif, R. Lee, and M. J. Kochenderfer, "Adaptive stress testing for autonomous vehicles," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 1–7.

[16] Y. Abeysirigoonawardena, F. Shkurti, and G. Dudek, "Generating adversarial driving scenarios in high-fidelity simulators," in *Proc. IEEE*



Top: from above at the failure, in metres, ego (red) driving to the right, the car it met orange, others grey, the human's recorded path dashed; lines from the hand-off on. Below, to 1.5 s after the failure: dotted, the policy takes over; solid, the failure.

Fig. 6. Three confirmed hardest cases, from above and as speed and gap over time.

International Conference on Robotics and Automation (ICRA), 2019, pp. 8271–8277.

[17] O. Osikowicz, P. McMinn, W. Xing, and D. Shin, "Multi-fidelity Bayesian optimization for simulation based autonomous driving systems testing," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2026. [Online]. Available: https://eprints.whiterose.ac.uk/id/eprint/237186/

[18] Y. Mei, T. Nie, J. Sun, and Y. Tian, "LLM-Attacker: Enhancing closed-loop adversarial scenario generation for autonomous driving with large language models," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 10, pp. 15 068–15 076, 2025.

[19] Z. Ghazal, K. Gaaloul, and B. Maxim, "Teach-to-Crash: A closed-loop student-teacher LLM framework for collision-inducing test scenario generation," in *Proc. AgenticDev Workshop, 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026.

[20] T. Nie, Y. Mei, Y. Tang, J. He, J. Deng, J. Sun, and W. Ma, "EvoDrive: Pareto evolution for safety-critical autonomous driving via self-improving LLM agents," in *Proc. Conference on Robot Learning (CoRL)*, 2026, arXiv:2606.03678.

[21] Y. Gao, S. Müller, M. Piccinini, M. Kaufeld, Y. Zhang, F. R. Schäfer, Q. Song, and J. Betz, "PlannerForge: LLM agents for scenario-based testing of motion planners in autonomous driving," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026, to appear. arXiv:2609.08965.

[22] H. Tian, X. Han, Y. Zhou, G. Wu, A. Guo, M. Cheng, S. Li, J. Wei, and T. Zhang, "LMM-enhanced safety-critical scenario generation for autonomous driving system testing from non-accident traffic videos," 2024, arXiv:2406.10857.

[23] Y. Zhao, W. Xiao, T. Mihalj, J. Hu, and A. Eichberger, "Chat2Scenario: Scenario extraction from dataset through utilization of large language model," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2024, pp. 559–566.

[24] G. Pokharkar, S. Baravkar, and Z. Song, "EC-Detector: AI assisted edge case discovery for AV/ADAS vehicle testing," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2026.

[25] K. Poddubnyy, I. Vozniak, I. Burmistrov, N. Lipp, D. Hovhannisyan, C. Müller, and P. Slusallek, "ARISE – adaptive refinement and iterative scenario engineering," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2026, arXiv:2601.14743.

[26] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[27] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The Simplex architecture for safe online control system upgrades," in *Proc. 1998 American Control Conference (ACC)*, vol. 6, 1998, pp. 3504–3508.

[28] U. Mehmood, S. Sheikhi, S. Bak, S. A. Smolka, and S. D. Stoller, "The black-box Simplex architecture for runtime assurance of autonomous CPS," in *NASA Formal Methods (NFM 2022)*, ser. Lecture Notes in Computer Science. Springer, 2022, pp. 231–250, arXiv:2102.12981.

[29] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[30] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026, pp. 2938–2950, arXiv:2503.18666.

[31] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding agents via verifiable safety policy reasoning," in *Proc. 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 8313–8344, arXiv:2503.22738.

[32] S. Eshun, D. Murage, S. Muoki, C.-C. Chen, S. Adjignon, M. Staar, and O. Angélil, "Agentic self-healing for data and AI pipelines: An affordable vendor-agnostic architecture using open-source software," 2026, arXiv:2608.01955.

[33] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," 2024, arXiv:2408.06292.

[34] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.

[35] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.

[36] F. Bartoccioni, E. Ramzi, V. Besnier, S. Venkataramanan, T.-H. Vu, Y. Xu, L. Chambon, S. Gidaris, S. Odabas, D. Hurych, R. Marlet, A. Boulch, M. Chen, É. Zablocki, A. Bursuc, E. Valle, and M. Cord, "VaViM and VaVAM: Autonomous driving through video generative modeling," 2025, arXiv:2502.15672.

[37] Euro NCAP, "AEB/LSS VRU systems test protocol, version 4.5," European New Car Assessment Programme, 2023.

[38] ——, "AEB car-to-car systems test protocol, version 4.3," European New Car Assessment Programme, 2023.